

구현 브리프

K-PING 구현 브리프 (Claude Code 투입용)

이 문서 하나로 "뭘 만들지 + 뉴스는 어디서 가져올지 + 어떤 순서로 + 어떤 툴/API/MCP로" 구현 가능하도록 정리했다. 기존 기획안·기능명세서·API명세서를 압축해서 다시 담았으니, Claude Code에 이 파일 하나만 넘겨도 시작할 수 있다.

0. 서비스 한 줄 정의

지도 위에 한류 뉴스를 핀으로 표시하고(발생 핀/방송·촬영 핀/반응 핀), 핀을 탭하면 AI가 만든 다국어 쇼폼을 스와이프로 소비하는 서비스.

1. 뉴스는 실제로 어디서 가져오나 (검증된 소스)

1순위 — 네이버 뉴스 검색 오픈API (공식, 가장 안정적)

```
GET https://openapi.naver.com/v1/search/news.json?query={검색어}&display=100&sort=date
Header: X-Naver-Client-Id, X-Naver-Client-Secret
```

- developers.naver.com에서 애플리케이션 등록 → Client ID/Secret 즉시 발급 (무료)
- **일일 25,000회 호출 한도** — RSS 폴링보다 훨씬 구조화된 JSON 응답(제목/링크/설명/발행일)을 줌
- `query`에 "아이돌 컴백", "케이팝 콘서트", 아티스트명 등을 넣어 검색 기반으로 수집. RSS처럼 소스 하나를 구독하는 게 아니라, **검색어 조합으로 여러 매체를 한 번에 커버**하는 방식이라 K-PING 구조에 더 잘 맞춤
- 응답의 `link`가 실제 언론사 원문 링크라, 여기서 본문을 재크롤링해야 함 (검색 API는 요약만 줌)

2순위 — 개별 언론사 공식 RSS (직접 확인됨)

언론사	연예 섹션 RSS	비고
뉴스시스	https://www.newsis.com/RSS/entertain.xml	통신사라 속보성 좋음
한국경제	https://www.hankyung.com/feed/entertainment	

- OSEN·스타뉴스·텐아시아·마이데일리 등 연예 전문지는 **공개 RSS를 더 이상 제공하지 않는 경우가 많음** — 확인 안 된 URL을 임의로 쓰지 말고, 소스 등록 전 반드시 `curl`로 실제 XML이 오는지 확인부터 할 것
- 새 언론사 RSS를 찾으면 `/admin/sources`에 등록만 하면 파이프라인이 그대로 재사용됨 (기획안 2장 구조)

3순위 — Google News RSS 검색 (보조/확장용, 커버리지 넓힘)

```
https://news.google.com/rss/search?q={검색어}&hl=ko&gl=KR&ceid=KR:ko
```

- 국내 매체 전체를 검색어 기반으로 커버 — 1순위 API의 일일 한도를 넘기거나 특정 매체가 안 잡힐 때 보조용
- 결과가 구글 리다이렉트 링크라 원문 URL 추출에 한 단계(리다이렉트 따라가기)가 더 필요함

4순위 — 방송/촬영 핀 전용 소스

- 방송사(MBN, SBS, KBS, MBC)는 편성표 공식 API가 없음. 각 방송사 홈페이지의 프로그램 편성 페이지를 주기적으로 스크래핑하거나, 초기엔 **운영자가 수동으로 촬영/녹화 일정을 입력하는 방식으로 시작**하는 게 현실 적 (기획안 8장의 "없으면 아무것도 못 함" 항목과 별개로, 이건 MVP 이후로 미뤄도 됨)

5순위 — 팬덤 플랫폼 (위버스/버블/블립)

- 공식 공개 API 없음. 약관상 자동 수집이 명확히 허용되지 않으므로 **파트너십 체결 전까지는 소스 등록을 보류** 한다 (기능명세서 2장의 `license_status: unreviewed` 규칙 그대로 적용)

2. 프로젝트 구조 (Java Spring Boot 기준, Claude Code가 스캐폴딩할 기준)

```
k-ping/
├── docker-compose.yml
├── .env.example
├── backend/
│   ├── build.gradle (또는 pom.xml)
│   ├── src/main/java/com/kping/
│   │   ├── KPingApplication.java
│   │   └── api/                                # @RestController (MapController, FeedController, EventController, AdminC
ontroller)
│   │   ├── domain/                            # JPA 엔티티 (Source, RawArticle, Event, EventLocation, EventEntity, Shor
t, Artist, User, RenderJob)
│   │   ├── repository/                        # Spring Data JPA (필요시 QueryDSL로 공간·벡터 질의 native query)
│   │   ├── dto/                              # 요청/응답 DTO (record 클래스 권장)
│   │   ├── service/                          # 비즈니스 로직
│   │   └── worker/                            # 파이프라인 단계별 처리기
│   │       ├── IngestWorker.java             # A-1 수집
│   │       ├── ClusterWorker.java            # A-2 클러스터링
│   │       ├── AnalyzeWorker.java            # A-3 팩트정리
│   │       ├── LocalizeWorker.java           # A-4 다국어 현지화
│   │       ├── GeotagWorker.java             # A-5 지오태깅
│   │       └── RenderWorker.java              # A-6 영상 렌더링
│   └── client/                               # 외부 API 클라이언트 (WebClient 기반)
│       ├── NaverNewsClient.java
│       ├── ClaudeClient.java
│       ├── VoyageEmbeddingClient.java
│       ├── KakaoGeocodingClient.java
│       └── TtsClient.java
│   └── config/                               # WebClient 빈, 스케줄러, 큐 설정
├── src/main/resources/
│   ├── application.yml
│   └── db/migration/                         # Flyway SQL (V1__init.sql ...)
├── frontend/
│   └── (Next.js + MapLibre GL 앱)
└── media-templates/                         # FFmpeg용 영상 템플릿, 폰트, BGM, B-roll
```

기존 기획서의 FastAPI+Celery 조합을 Spring Boot 기준으로 바꾼 매핑

기존 (Python)	Java Spring 대체
FastAPI	Spring Boot (Web MVC 또는 WebFlux)
SQLAlchemy + Alembic	Spring Data JPA + Flyway
Celery + Redis (큐)	아래 3장 "큐 전략" 참고
Pydantic 스키마	DTO(record) + Bean Validation
requests / httpx	WebClient (Spring WebFlux의 HTTP 클라이언트)

3. 큐 전략 (Celery 대체)

Spring 생태계에서 파이프라인 단계를 분리하는 방법은 두 가지가 있다. 팀 규모와 일정에 맞춰 고르면 된다.

방식	언제 쓰나	구현
DB 폴링 방식 (MVP 추천)	1~4주 짧은 개발 기간, 인프라 단순화 우선	<code>render_jobs</code> 류 테이블에 <code>status(queued/running/succeeded/failed)</code> 컬럼 두고, <code>@Scheduled</code> + <code>ThreadPoolTaskExecutor</code> 로 주기적으로 <code>queued</code> 건을 집어서 처리. 이미 기획서 5.4의 상태 전이(<code>collected</code> → <code>clustered</code> → <code>analyzed</code> → ...) 모델과 그대로 맞아떨어짐
RabbitMQ (정식 확장 시)	처리량이 늘거나 단계별 워커를 별도 프로세스로 분리 배포하고 싶을 때	<code>spring-boot-starter-amqp</code> . 단계별 큐(<code>ingest.queue</code> , <code>cluster.queue</code> ...)를 분리하고 재시도/데드레터도 기본 제공

이미 익숙한 **Redis**를 캐시(지도 조회, 세션)로 쓰면서, 큐는 굳이 Redis Streams로 억지로 만들기보다 위 두 방식 중 하나로 가는 게 깔끔하다.

4. 구현 순서 (단계별 실제 작업)

Step 1. 인프라 스케폴딩

- `docker-compose.yml` : Spring Boot 앱, Postgres(+PostGIS+pgvector 익스텐션), Redis (+ RabbitMQ, 정식 확장 시)
- Flyway로 `sources / raw_articles / events / event_locations / event_entities / shorts / artists / users / render_jobs` 테이블 생성 (기존 API명에서 데이터 모델 그대로)
- pgvector는 JPA가 기본 지원 안 하므로, 벡터 컬럼은 `@Column(columnDefinition = "vector(1536)")` + native query(`ORDER BY embedding <=> :queryVector`)로 처리

Step 2. 뉴스 수집 (A-1)

- `NaverNewsClient` : `WebClient` 로 네이버 뉴스 검색 API 호출 → 키워드 리스트(아티스트명, "컴백", "콘서트" 등)를 순회하며 `raw_articles` 에 적재
- 본문 추출: 검색 API는 요약만 주므로 원문 URL을 다시 가져와야 함. Java는 **Jsoup**으로 HTML 파싱 + 본문 추출 (언론사별 CSS 셀렉터가 달라서 도메인별 파서 매핑 필요, 또는 범용 추출 라이브러리인 `boilerpipe` 류 검토)
- 중복 방지: 원문 URL 해시(`raw_articles.content_hash` UNIQUE 제약)로 `last_seen_at` 만 갱신 — 여기는 이미 은행 프로젝트에서 쓴 UNIQUE 제약 기반 idempotency 패턴 그대로 재사용 가능

Step 3. 클러스터링 (A-2)

- 임베딩: **Anthropic**은 임베딩 API 미제공이라 외부 API가 필요함
 - **Voyage AI API 호출을 기본으로 추천** — Java 생태계에서 임베딩 모델을 직접 서빙(ONNX/DJL)하는 건 Python보다 훨씬 손이 많이 감. Spring 스택 유지하려면 `VoyageEmbeddingClient` (WebClient)로 호출하는 게 제일 깔끔함
 - 굳이 비용을 아끼고 싶으면 임베딩 계산만 별도 Python 마이크로서비스(FastAPI 한 파일 정도)로 분리하는 방법도 있지만, 초기엔 API 호출로 시작하는 걸 권장
- pgvector 코사인 유사도로 72시간 내 이벤트와 비교 (native query), 임계값 로직은 `ClusterWorker` 에서 처리

Step 4. 팩트정리 + 다국어 현지화 (A-3, A-4)

- **Anthropic Claude API** (`claude-sonnet-5` 또는 비용 절감형 `claude-haiku-4-5`) — `ClaudeClient` 에서 WebClient로 직접 호출하거나 Anthropic 공식 Java SDK 사용
- 구조화 출력이 핵심: Claude의 `tool_use`(함수 호출) 기능으로 JSON 스키마를 강제하고, Jackson으로 바로 DTO 역직렬화
- 언어별 호출은 반드시 분리 (기획서 원칙: 번역이 아니라 재작성)

Step 5. 지오태깅 (A-5)

- 한국 장소 지오킨코딩은 **카카오 로컬 API** 추천 — 국내 장소·랜드마크 정확도가 좋고 무료 할당량 넉넉함, `KakaoGeocodingClient` 로 래핑
- 해외 장소(투어 등) 폴백용으로 Google Geocoding API 병행

Step 6. 영상 렌더링 (A-6)

- TTS: **ElevenLabs**(다국어 품질 좋음) 또는 **Google Cloud TTS**(언어 커버리지 넓고 저렴) 중 예산에 맞게 선택. 한국어 특화 중요하다면 **네이버 클로바보이스**도 검토
- 합성: Java에서 FFmpeg 바이너리를 `ProcessBuilder` 로 직접 호출 (Python처럼 전용 바인딩 라이브러리를 쓸 필요 없이, `ffmpeg -filter_complex ...` 커맨드를 그대로 조립해서 실행하면 됨). 표준 출력/에러 스트림을 로그로 남겨서 실패 원인 추적
- 이미지 소재 없을 때 폴백: `java.awt.Graphics2D` 또는 Thumbnailator 같은 라이브러리로 그라디언트+타이포그래피 카드 생성

Step 7. 프론트엔드

- Next.js + MapLibre GL JS로 지도, TanStack Query로 피드 무한스크롤 (프론트는 백엔드 언어와 무관하게 기존 계획 그대로)
- API명세서의 엔드포인트를 그대로 연동

Step 8. 운영 도구

- `/admin/sources` , `/admin/events` 컨트롤러 구현 — 소스 등록과 검토 큐 승인/반려

5. 역할별 툴/API 정리표

역할	추천	비고
백엔드 프레임워크	Spring Boot 3.x (Web MVC)	이미 익숙한 스택 그대로
ORM / 마이그레이션	Spring Data JPA + Flyway	Hibernate 기반, 기존 경험 재사용
DB	PostgreSQL + PostGIS + pgvector	벡터 컬럼은 native query로 처리
큐	DB 폴링(<code>@Scheduled</code> , MVP) 또는 RabbitMQ(확장 시)	3장 참고
HTTP 클라이언트	Spring WebClient	외부 API(네이버/Claude/카카오 등) 호출
뉴스 수집	네이버 뉴스 검색 오픈API + 개별 RSS + Google News RSS	1~3순위 조합
본문 추출	Jsoup	HTML에서 본문만 뽑는 파서
임베딩	Voyage AI API	Anthropic은 임베딩 미제공, Java에서 자체 서빙보다 API가 현실적

역할	추천	비고
요약/현지화/팩트정리	Anthropic Claude API	구조화 출력은 tool_use로 강제, Jackson으로 역직렬화
지오코딩	카카오 로컬 API (국내) + Google Geocoding (해외 폴백)	
TTS	ElevenLabs 또는 Google Cloud TTS	다국어 품질 기준으로 선택
영상 합성	FFmpeg (<code>ProcessBuilder</code> 로 직접 실행)	
프론트엔드	Next.js + MapLibre GL + TanStack Query	백엔드 언어와 무관
인프라	Docker Compose (로컬/데모), GitHub Actions (CI)	기존 SDG-BACKEND 프로젝트 경험 재사용 가능

6. Claude Code 개발 중 활용할 MCP

MCP	용도
Naver Search MCP (<code>isnow890/naver-search-mcp</code> , GitHub 공개)	개발 중 네이버 뉴스 검색 API 응답을 Claude Code가 직접 호출·검증하면서 파서 로직을 짤 수 있음. <code>NAVER_CLIENT_ID</code> / <code>SECRET</code> 환경변수만 설정하면 바로 연결
GitHub MCP	이슈 생성, PR 리뷰, 브랜치 작업을 Claude Code가 레포와 직접 연동해서 처리
Postgres MCP (커뮤니티 제공, 예: <code>crystaldba/postgres-mcp</code>)	로컬 DB 스키마 확인, 쿼리 테스트, Flyway 마이그레이션 검증을 Claude Code가 직접 실행하며 디버깅
Filesystem MCP	Claude Code 자체에 기본 내장 — 별도 설정 불필요
Playwright MCP (팬덤 소스 붙일 때, 파트너십 확정 후에만)	동적 렌더링 페이지 크롤링 테스트용. 현재는 팬덤 소스 보류 상태라 우선순위 낮음

MCP는 전부 "개발자(너)가 Claude Code로 작업할 때" 쓰는 도구다. 서비스 자체(K-PING 백엔드)가 MCP를 호출하는 구조는 아니다 — 서비스는 위 표의 REST API들을 직접 호출한다. Naver Search MCP는 TypeScript로 만들어졌지만 백엔드가 Java든 Python이든 상관없이 Claude Code 쪽 개발 도구로만 쓰인다.

7. 환경변수 체크리스트 (`.env.example` / `application.yml` 에 넣을 것)

```
# DB
DATABASE_URL=postgresql://...
REDIS_URL=redis://...

# 뉴스 수집
NAVER_CLIENT_ID=
NAVER_CLIENT_SECRET=

# AI
ANTHROPIC_API_KEY=
VOYAGE_API_KEY=

# 지오코딩
KAKAO_REST_API_KEY=
GOOGLE_GEOCODING_API_KEY=

# TTS
ELEVENLABS_API_KEY=
# 또는 GOOGLE_TTS_CREDENTIALS_JSON=

# 스토리지/CDN
S3_BUCKET=
S3_ACCESS_KEY=
```

```
S3_SECRET_KEY=  
CDN_BASE_URL=
```

8. Claude Code에 넘길 때 프롬프트 예시

이 문서(K-PING_구현브리프.md)와 K-PING_API명세서.md, K-PING_기능명세서.md를 참고해서 Step 1(인프라 스캐폴딩)부터 시작해줘. Spring Boot 3.x + Gradle 기준으로, docker-compose.yml, Flyway 초기 마이그레이션, 프로젝트 뼈대를 먼저 만들고 Step 2(네이버 뉴스 API 연동)로 넘어가자.

문서 4개(기획안 v2, 기능명세서, API명세서, 이 구현브리프)를 같은 폴더에 두고 통째로 넘기면 Claude Code가 맥락을 놓치지 않는다.